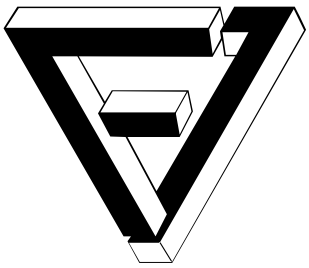
Masaryk University
Faculty of Informatics



Side-Effect Analysis Using a Static Single Use Program Representation

Master's Thesis

Samuel Malec

Brno, 2026

Declaration

Hereby I declare that this thesis is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

During the preparation of this thesis, I used Claude Code (Anthropic, `https://claude.ai`) to write and review source code, to run and summarise experiments, and to help structure the text. After using this tool, I checked the content and accept full responsibility for it.

Supervisor: Petr Ročkai

Acknowledgements

Write your acknowledgements here.

Abstract

Write your abstract here.

Keywords

side-effect analysis, static single use, linear types, points-to analysis, ahead-of-time compilation, memory optimization, thread escape analysis, lock elision, GraalVM Native Image

Contents

1 Introduction 1

2 Background 2
- 2.1 Side Effects and Their Analysis . . . 2
- 2.2 Program Representations . . . 2
- 2.3 Graal and Its Memory Model . . . 2
- 2.4 Native Image . . . 2

3 Analysis of the Problem 3
- 3.1 What a Call Costs the Optimizer . . . 3
- 3.2 What Information Would Help . . . 3
- 3.3 Requirements . . . 3
- 3.4 Approach . . . 3

4 Explicit Effects 4
- 4.1 A Language with Heap Tokens . . . 4
- 4.2 Semantics . . . 4
- 4.3 Partitions . . . 4
- 4.4 Signatures . . . 4
- 4.5 What the Tokens Guarantee . . . 4
- 4.6 Exceptions and Deoptimization . . . 4
- 4.7 Concurrency . . . 4
- 4.8 From Signatures to Kill Sets . . . 4

5 Implementation 5
- 5.1 Architecture . . . 5
- 5.2 Exporting the Points-to Analysis . . . 5
- 5.3 Translation to the Token Form . . . 5
- 5.4 Signatures and Kill Sets . . . 5
- 5.5 Back into Graal . . . 5
- 5.6 Thread Escape in Native Image's Analysis . . . 5

5.7 Lock Elision and Ordinary Volatile Accesses . . . 5

6 Evaluation 6
6.1 Method . . . 6
6.2 What Graal Optimizes with the Facts (RQ1) . . . 9
6.3 Cost against Benefit (RQ2) . . . 12
6.4 What Modelling Concurrency Enables (RQ3) . . . 13
6.5 Soundness in Practice (RQ4) . . . 16
6.6 What Blocks the Analysis . . . 16
6.7 Discussion . . . 17
6.8 Threats to Validity . . . 18

7 Related Work 20
7.1 Side-Effect and Mod/Ref Analyses . . . 20
7.2 Memory in Compiler Representations . . . 20
7.3 Escape Analysis and Synchronization Removal . . . 20
7.4 Analyses for Ahead-of-Time Compilation . . . 20

8 Conclusion 21
8.1 Results . . . 21
8.2 Contribution . . . 21
8.3 Future Work . . . 21

Bibliography 22

A Electronic Attachments 23

B Proofs 24

C The Dialect of Graal IR in Cthulhu 25

D Reproducing the Results 26
D.1 The Headline Tables from One Commit . . . 26
D.2 Other Measurements . . . 26

I Introduction

2 Background

2.1 Side Effects and Their Analysis

2.2 Program Representations

2.3 Graal and Its Memory Model

2.4 Native Image

3 Analysis of the Problem

3.1 What a Call Costs the Optimizer

3.2 What Information Would Help

3.3 Requirements

3.4 Approach

4 Explicit Effects

4.1 A Language with Heap Tokens

4.2 Semantics

4.3 Partitions

4.4 Signatures

4.5 What the Tokens Guarantee

4.6 Exceptions and Deoptimization

4.7 Concurrency

4.8 From Signatures to Kill Sets

5 Implementation

5.1 Architecture

5.2 Exporting the Points-to Analysis

5.3 Translation to the Token Form

5.4 Signatures and Kill Sets

5.5 Back into Graal

5.6 Thread Escape in Native Image's Analysis

5.7 Lock Elision and Ordinary Volatile Accesses

6 Evaluation

This chapter evaluates the facts that our analysis gives to Graal when it compiles a program with GraalVM Native Image. It answers the four research questions of Chapter 1: how much more Graal optimizes with the facts (RQ1), what this costs in build time (RQ2), what modelling concurrency enables (RQ3), and whether the resulting programs are correct (RQ4). Section 6.1 states what is compared with what and under which conditions. Sections 6.2 to 6.5 answer the questions in turn, Section 6.6 explains why the gains are as small as they are, and Sections 6.7 and 6.8 interpret the results and state their limits. The commands that regenerate every table are in Appendix D.

6.1 Method

6.1.1 What Is Compared with What

An *image* is a native image built from one program by one builder. The builder is Native Image from our fork of Graal, extended with the feature of Chapter 5. The feature takes the facts of Native Image's own points-to analysis out of the build, our tool turns them into kill sets, partitions, and lock decisions, and Graal's own phases use them (Section 5.5). Recall that a *kill set* is the set of locations a call may write: without one, Graal assumes that the call may write anything and reloads every field after it. A *configuration* says which of these facts reach Graal. Table 6.1 lists the configurations used in this chapter.

Native Image can run its points-to analysis in two ways. The *default analysis* is context insensitive and names an object by its type. The *allocation-site analysis*, selected by setting the builder's option `AnalysisContextSensitivity` to `allocsens`, names an object by the place that allocated it. Partitions by object need the finer names: on the default analysis an object is a type, so they equal the partitions by field and there is nothing to compare. Kill sets do not need them, so we measure them on both. Thread escape and lock elision work on both; allocation sites find more sites (43 against 34 thread-local sites in AWFY), and all our run-time measurements of lock elision use them, so we state this choice in Section 6.7 and Section 6.8.

Allocation sites are not only a more precise input; they change the code Graal produces, with or without our facts. On Are We Fast Yet [2] (AWFY below), with nothing of ours narrowed, the

Table 6.1: The configurations. Each is compared with the baseline that differs from it in exactly the facts under test. `graal` loads the feature but hands nothing to Graal; it only counts what Graal leaves, so apart from the counting it is Native Image as released. `ceiling` leaves out the kill of everything the facts cannot name and is therefore unsound; it is an upper bound, not a result. Allocation sites stand for the allocation-site analysis explained below. `kills` is also built on allocation sites, where its baseline is `graal` on allocation sites.

Configuration	Analysis	Facts given to Graal	Compared with
`graal`	default	none	(baseline)
`kills`	default	kill sets	`graal`
`ceiling`	default	kill sets, unsound upper bound	`graal`, `kills`
`graal`	allocation sites	none	(baseline)
`objects`	allocation sites	kill sets and object-field partitions	`graal` on allocation sites
`locks`	allocation sites	lock and `volatile` decisions	`graal` with the same flags

calls left in the program's methods fall from 1,407 to 1,110, and the loads Graal leaves in the build a user would make rise from 2,564 to 3,325 (+30 %). An `objects` image with 3,270 loads, set beside a default-analysis `graal` image with 2,564, would therefore look 28 % worse; in the other build (914 loads against 943) it would look 3 % better. Neither number says anything about partitions. An early run-time measurement fell into exactly this trap, showing DeltaBlue 18.8 % faster and Havlak 14.4 % slower; the change of analysis confounded it, and we discard it. We therefore compare images only on the same analysis, and report the price of the allocation-site analysis separately (Section 6.3).

6.1.2 Programs

Table 6.2 lists the programs. The first four serve the questions that need a known answer, AWFY the questions about run time, and javac, PetClinic, and the Barista applications the questions about size and about locks that real programs own. AWFY has 14 benchmarks over a small collection library of their own, so the JDK stays out of most of the code. The suite's harness picks a benchmark by name through reflection, which a closed-world analysis cannot see; `Awfy.java` therefore constructs each benchmark directly and prints whether it verified.

6.1.3 Conditions

All builds and measurements ran on the faculty server pontos04: two AMD EPYC 7371 processors with 64 threads in total, 1 TB of RAM, Debian 13. It is the only machine of the evaluation; a laptop (Ryzen 7 PRO 7840U) served only for the early work with hardware counters and for counts that do not depend on the machine. Hardware counters are not available on pontos04 (`perf_event_paranoid` is 3). The builder is GraalVM 25.5-dev, built from our fork of Graal

Table 6.2: The programs of the evaluation. Methods are counted as the analysis sees them: reachable methods for javac, methods with facts for PetClinic.

Program	What it is	Used for
`Examples`, `Signatures`, `Aliasing`, `Allocation`	four small programs designed to have a known answer	loads
Example suite	72 one-concept programs, 32 of them on concurrency	round trips, checks
AWFY	14 benchmarks, 459 methods of the program, pinned at commit `74306fe`	loads, run time, cost
`LockShapes`, `VolatileShapes`	a lock or `volatile` field on an object held in a field, an array, a list, or passed through calls	run time of lock elision
javac	the Java compiler, 37,854 reachable methods	cost, locks
Native Image's tests	its own JUnit suite, 23 images with 289 tests	robustness
Spring PetClinic	a server with Tomcat, H2, HikariCP, 164,600 methods with facts	scale, locks
Barista [1]	13 server applications taken, 11 built (Vert.x, Micronaut, Spring, Helidon, Ktor, Play, Dropwizard, …)	locks

on a JDK 25 with JVMCI. The headline tables of this chapter come from two runs of a script (`reproduce.sh`, Appendix D) that refuses to start unless both trees are exactly a commit and the builder is newer than the fork's sources; every result directory records the commits. The first run, of 29 October, used commit `bc9abea792a` of the fork and `622fc27` of our tool, the second, of 31 October, `bcce7f21bfe` and `e56d77f`. The only change to the builder between them is that the exported graphs are written one by one. The tables give both runs wherever they differ. The measurements of 22–28 October were made from stamped working copies of earlier commits. We say so wherever a number comes from one of them.

Programs are built twice. AWFY is built as the analysis builds it (`-O2`, no trivial inlining) and as a user would (`-O3` with trivial inlining). We call the two builds `-O2` and `-O3`.

Correctness. Every image must print exactly what the JVM prints for the same program. An image that runs longer than 300 seconds is stopped. A build whose output differs is a failure, not a data point.

What is counted. We count the field loads and stores that Graal leaves in the program's methods at the end of the low tier, the calls, and the calls narrowed. These are static counts: loop unrolling multiplies them, so the 66 stores of `Allocation` are one store unrolled 64 times. They are not run time.

Run time. The images are pinned to four cores of one NUMA node (`taskset -c 4-7`) with nothing else running; the script waits until the one-minute load is below 2. Each of the 14 benchmarks runs 20 iterations, five times; the first iteration is dropped as warm-up and the median is the result. The images take turns run by run, so that drift touches them alike. In every table the same `graal` binary is timed twice, as an A/A control that shows the noise: in the three sets of Section 6.2 it was between −2.8 % and +1.5 % per benchmark (the −2.8 % is DeltaBlue, which runs for 15 ms) and between −0.2 % and +0.1 % in total.

Several builds. Native Image places every method at a multiple of 32 bytes, so a change in the size of one method moves all that follow it. Placement alone moves twelve of the fourteen AWFY benchmarks by 4–13 % with identical instructions, and the effect of our facts is below that for all but one benchmark. One build of a configuration is therefore not enough, and we build `graal` and `kills` four times each and time all eight images together. We did this three times: once in a session of 29 October before the first clean run, and once in each clean run. That makes twelve builds of `kills`.

Build time. We report the wall-clock seconds of one build at a time on an empty machine, the configurations taking turns: five rounds for AWFY, three for javac. The tables give the median and the range, the time of the analysis and compile stages from the builder's own log, and the peak resident memory.

6.2 What Graal Optimizes with the Facts (RQ1)

6.2.1 Loads Left

Table 6.3 shows the loads Graal leaves in the program's methods. The numbers are those of the clean runs of 29 and 31 October, which agree to the unit; they equal the earlier ones from 22–28 October.

On the four small programs, which were written to contain calls that write little, kill sets remove 12.5–32.5 % of the loads. On AWFY they remove 40 of 943 loads (4.2 %) in the `-O2` build and 54 of 2,564 (2.1 %) in the `-O3` build. The effect of kill sets does not depend on the analysis: on allocation sites they remove the same 40 loads from AWFY. Partitions by allocation site add nothing on AWFY, where `objects` leaves exactly the loads that `kills` leaves, because the partitions split almost no field of the program (one of 234 fields keeps two partitions). They pay only where the program was written for them: on `Aliasing`, which holds two counters that never meet, they remove three loads more than `kills` does (16 to 13). Dead stores add nothing of their own: Graal's

Table 6.3: Loads left by Graal in the program's methods, by configuration. The first block uses the default analysis, the second allocation sites; the percentage is the change against `graal` on the same analysis. For AWFY the build is `-O2` unless stated. Rows marked with an asterisk were not repeated in the clean runs: the `-O3` build on allocation sites is from 22 October and the ceiling at `-O2` from 28 October. The third block splits the default-analysis loads of the four small programs, `graal` to `kills`, by the kind of method. *Controls* are the methods named `...Control`: a call in them may write the field, or a reference may alias another, so they must keep their loads. *Unchanged* are the loads of the other methods that do not change (helpers, `main`, the inlined monitor code of a synchronized method, and in `Signatures` the three methods that cross a monitor, a native call, and a `volatile` access). *Reduced* are the loads of the methods that lose loads. The totals in the first two blocks count every method; the three groups add up to them.

	Default analysis		Allocation sites			By kind of method		
Program	`graal`	`kills`	`graal`	`kills`	`objects`	Controls	Unchanged	Reduced
`Examples`	40	27 (−32.5 %)	43	30 (−30.2 %)	30 (−30.2 %)	16 → 14	10	14 → 3
`Signatures`	16	12 (−25.0 %)	16	12 (−25.0 %)	12 (−25.0 %)	–	12	4 → 0
`Aliasing`	16	14 (−12.5 %)	18	16 (−11.1 %)	13 (−27.8 %)	5 → 4	9	2 → 1
`Allocation`	9	7 (−22.2 %)	9	7 (−22.2 %)	7 (−22.2 %)	–	0	9 → 7
AWFY, `-O2`	943	903 (−4.2 %)	954	914 (−4.2 %)	914 (−4.2 %)	–	–	–
AWFY, `-O3`	2,564	2,510 (−2.1 %)	3,325*	3,269* (−1.7 %)	3,270* (−1.7 %)	–	–	–

phase for overwritten stores removes the same stores as our decisions did (three in `Examples`, one in `Aliasing`).

6.2.2 Run Time

A load that Graal no longer reloads after a call saves nothing by itself; it saves time where it enables something more, such as a load moved out of a loop. We therefore measure run time directly. Table 6.4 gives the benchmarks that matter at `-O3` with the default analysis. Each `kills` cell is a range over twelve builds in three sets of four: one timed earlier on 29 October, one from the clean run of 29 October, and one from the clean run of 31 October. Each build is compared with the first `graal` build of its set.

Queens is the one clear win: −25.7 % to −27.8 % in all fifteen builds of `kills` of 29 and 31 October (the twelve of the table and three more, from the test run and from the two ceiling stages), far outside the noise. The recursive method `placeQueen` now kills only array elements, so four fields are no longer reloaded in its loop (the reads in the loop fall from 6 to 0) and Graal inlines the recursion differently. The win reproduces on allocation sites (−32.4 %) and is absent in the `-O2` build (−0.5 %), where Graal inlines less. We read it as the effect we set out to obtain: a call that wrote a few locations no longer forces a reload of the others.

The total needs care. The first four builds of `kills` were 1.4 to 3.0 % faster than `graal` (mean −2.3 %); the four of the first clean run gave +1.7 %, −1.3 %, −1.0 %, and −1.0 % (mean −0.4 %),

Table 6.4: Run time of `kills` against `graal` on AWFY at `-O3`, default analysis. The first column is the A/A control: the first `graal` binary of a set timed against itself, as a range over the three sets. The second is the range over the nine other builds of `graal`, three in each set, the third the range over the twelve builds of `kills`. A negative number is faster. The first `graal` build took 704.6 ms in total in the first set, 696.7 ms in the first clean run and 706.6 ms in the second; the other eight benchmarks stay within ±4.8 % for `kills`, the −4.8 % being DeltaBlue.

Benchmark	A/A	other `graal` builds	`kills`, twelve builds
Queens	−0.2 % to +0.1 %	−2.2 % to +0.9 %	−27.8 % **to** −25.7 %
Sieve	−0.9 % to +0.3 %	−2.0 % to +2.5 %	−8.6 % to −1.7 %
Storage	−0.0 % to +0.7 %	−0.0 % to +1.0 %	−5.4 % to −3.7 %
Permute	−0.6 % to +0.3 %	−1.7 % to +1.1 %	+5.2 % to +8.4 %
Havlak	−1.3 % to +1.5 %	−2.2 % to +1.5 %	−5.7 % to +2.5 %
Richards	−0.6 % to +0.1 %	−9.3 % to +6.7 %	−6.9 % to +20.9 %
Total	−0.2 % to +0.1 %	−1.6 % to +1.0 %	−3.0 % to +1.7 %

and the four of the second −1.5 %, −2.3 %, −1.8 %, and −2.1 % (mean −1.9 %). The +1.7 % is Richards alone: in that build Richards was 20.9 % slower, in the other three between −2.6 % and +4.6 %, and its share of that build's total is 3.0 points. Over the twelve builds the mean is −1.6 %, eleven of them are faster than `graal`, and the range is −3.0 % to +1.7 %, against −1.6 % to +1.0 % for the other builds of `graal`; the −1.6 % is a build of `graal` that Richards makes faster, by 9.3 %. We conclude that kill sets make AWFY 0 to 3 % faster in total, mostly through Queens, and that the number depends on which build is timed. An earlier reading of one build said "neutral in total" and the first four builds said −2.3 %; the range over twelve builds is the claim we can support.

Permute, Richards, and Havlak do not support a claim either way. Permute is slower with `kills` in every build (+4.7 % to +13.1 % over fifteen), but the kill sets change its code only a little, and when we shifted the alignment of all methods to 16, 64, or 128 bytes instead of the default 32, `kills` became 6.5–7.1 % *faster* than `graal`. We read it as code placement. Richards and Havlak vary by 28 and 8 points between builds of one configuration. With allocation sites in every image, the total is −0.1 % for `kills` and −0.6 % for `objects` at `-O3` (22 October): partitions by allocation site give no run-time gain on AWFY either.

6.2.3 The Ceiling

To learn whether making the remaining opaque calls precise would help, we built AWFY with the kill of everything the facts cannot name left out: identity hash codes, class initialization, synchronization, native and unresolved calls. This `ceiling` image is unsound by construction, but its output equals the JVM's on AWFY, and it is the most that precise kill sets could give Graal. It leaves 2,392 loads at `-O3` (−6.7 %) against 2,510 for `kills`, and 857 at `-O2` (−9.1 %): it removes three times as many loads. It is not faster. In one build each, `kills` was −1.7 % and `ceiling` −1.4 % in

Table 6.5: Build time of AWFY at -O3, median of five rounds in seconds, in the clean runs of 29 and 31 October. "Added" is the difference of the medians against the configuration the row adds to, with the percentage of its total. "Flags" means that the analysis keeps the inputs of its flows and the source positions, which `locks` needs. The compile stage is not shown: it is 7.2 to 7.8 seconds in every row and differs by at most 0.4 seconds from the row it adds to.

	29 October			31 October		
Configuration	Total	Analysis	Added	Total	Analysis	Added
`graal`	23.7	4.4		23.6	4.4	
`kills`	25.9	6.4	+2.2 (+9 %)	25.4	6.4	+1.8 (+8 %)
`graal` on allocation sites	31.9	11.9		31.0	11.3	
`objects` on allocation sites	33.1	14.3	+1.2 (+4 %)	34.3	15.2	+3.3 (+11 %)
`graal` with flags on allocation sites	30.9	10.7		31.3	11.6	
`locks` on allocation sites	33.4	14.3	+2.5 (+8 %)	33.4	14.1	+2.1 (+7 %)

total on 29 October, with Queens at −27.2 % in both; on 31 October `kills` was −1.9 % and `ceiling` −1.1 %, with Queens at −26.2 % and −25.3 %; a run on 28 October gave −1.7 % and −0.5 %. Only 19 methods differ between the two, and one of them, `LoopTesterApp.constructCFG`, accounts for 65 of the 118 further loads. As expected from Section 6.6, further precision removes loads that do not enable anything. Making the opaque layers precise would not make AWFY faster, so for AWFY it is not worth building.

6.3 Cost against Benefit (RQ2)

Table 6.5 gives the build time of AWFY at -O3 in the two clean runs: five rounds of every configuration in each. Each configuration is compared with the one it adds to on the same analysis and flags, as in Table 6.1; the input flows and source positions that thread escape needs cost Native Image nothing measurable by themselves (+0.1 s).

Kill sets add 1.8 to 2.2 seconds, 8 to 9 % of the build; lock elision adds 2.1 to 2.5 seconds, 7 to 8 %. Almost all of it is in the analysis stage, where the export of the facts takes 1.9 seconds (thread escape 0.2 s) and the tool 0.3 seconds; the compile stage does not move, and the peak memory stays between 5.2 and 5.6 GiB. The partitions are the one increment that the runs do not agree on: `objects` added 1.2 seconds (+4 %) in the first run and 3.3 seconds (+11 %) in the second, with the same steps of ours (export 2.4 seconds, tool 0.4). The rest is Native Image's own analysis, and the allocation-site analysis alone moves by 1–4 seconds between rounds, so we read the cost of the partitions as 4 to 11 %. The figures of 24 October were +7 % for kill sets and +4 % for lock elision, which is within that spread. On javac the kill sets add 10 seconds to 67 (+15 %) and kill sets with thread escape 14 seconds (+23 %) over the baseline that keeps the inputs of the flows (24 October, three rounds).

Table 6.6: Build time of `graal` on the default analysis and on allocation sites. AWFY gives the range of the two clean runs. javac is from 24 October, three rounds. For PetClinic the allocation-site build also keeps the input flows and source positions and limits the precision of saturated flows with a cutoff of 100; it is one build.

Program	Default analysis	Allocation sites	Factor
AWFY, total	23.6–23.7 s	31.0–31.9 s	1.31–1.35
AWFY, analysis stage	4.4 s	11.3–11.9 s	2.6–2.7
javac, total	67 s	288 s	4.3
javac, analysis stage	16.4 s	240 s	14.6
PetClinic, total	120 s	1,830 s	15

6.3.1 The Price of the Allocation-Site Analysis

The analysis that partitions by object and decides locks costs more than everything we add to it. Table 6.6 compares `graal` on the two analyses, with nothing of ours in either.

The price grows with the program, and it grows faster than the price of our facts. On javac the analysis stage with allocation sites takes 240 of 288 seconds and varies by 30 seconds between builds, more than anything we add, so the honest figure is our own steps: about 14.5 seconds for `objects` and 16.5 for `locks`, 5–6 % of the build. Part of the idea is cheap, and part of it is expensive, and the line between them is the analysis, not our tool: kill sets and, in principle, lock elision run on the default analysis, while partitions need the one that costs a multiple of the build, and our measurements of lock elision chose it too.

6.3.2 Frameworks

On PetClinic kill sets add 44–55 % to the build (118–122 seconds for `graal`, 176–183 for `kills`), because the export of 510 MB covers 164,600 methods. Lock elision with allocation sites and the cutoff took 2,395 seconds, of which the export was 284 (thread escape 191). On the Barista applications `kills` cost 1.12–1.49 times the build of `graal`, and the configuration with allocation sites and lock elision 3.0 to 10.7 times.

6.4 What Modelling Concurrency Enables (RQ3)

The concurrency model has two uses: it narrows the kill sets of calls that synchronize, and it lets thread escape decide which locks and `volatile` accesses only one thread can reach (Sections 4.7 and 5.6). We answer the question on designed programs first, then on real ones.

Table 6.7: Run time of two million calls of a shape, in milliseconds (the best of 30 repetitions in a run, then the best of seven runs, `-O3`). Each cell gives the clean run of 29 October and then that of 31 October. "Elision" is the `locks` image. The ratio is elision over `graal`.

Program	Shape	`graal`	Elision	Ratio
`LockShapes`	object in a field of another	28.37, 33.19	**1.77**, **1.77**	**0.06**, **0.05**
	object in an array	31.75, 31.74	31.78, 31.87	1.00, 1.00
	object in a list	33.09, 33.09	33.10, 33.09	1.00, 1.00
	object passed through calls	0.645, 0.645	0.645, 0.645	1.00, 1.00
	`StringBuffer`	6.22, 6.22	6.41, 6.47	1.03, 1.04
`VolatileShapes`	`volatile` field of an object in a field	14.73, 14.73	**1.77**, **1.77**	**0.12**, **0.12**
	`volatile` field of an object in an array	14.88, 14.88	14.88, 14.88	1.00, 1.00

6.4.1 Designed Shapes

The matrix of the 15 concurrency examples asks, for each, whether the object the program uses is shared by threads, under the assumptions that the virtual machine's own code and the root parameters publish nothing unnamed. The four plain cases—an object in a field of another, in an array of its own, an atomic, a `VarHandle`—are local under every analysis. A list and a queue are shared under every analysis, and so are the nine publications through a static field, a field, an array, a copy of an array, an atomic, a queue, a map, an executor, and a `VarHandle`. A matrix with all rows shared would show an analysis that is sound and useless; one with all rows local, an unsound one. This one has both kinds, and the controls that leave one kind of publishing site out flip exactly the examples that need it.

Where the answer is local, the lock or the `volatile` access can go. Table 6.7 gives the run time of the shapes in the two clean runs. Each shape has its own lock class and every image prints what the JVM prints.

A lock costs about 14 to 17 nanoseconds in this body, and a `volatile` write about 7; both go where thread escape proves the object local. The ratio of 16 to 19 is the ratio of a micro-benchmark whose only work is the lock, and we do not generalize it to programs. The range comes from one cell: `graal`'s image of the field shape took 28.37 milliseconds in the first run and 33.19 in the second (27.8 on 24 October), while the image without the lock took 1.77 each time. These are two builds of the same code; we did not establish why they differ by 17 %, and every other cell agrees within 0.1 milliseconds. The shapes that Graal already handles (a lock on a value passed through calls, a `StringBuffer`) are unchanged, and so are those that went through an array or a list, because the analysis cannot tell the elements apart.

Table 6.8: Locks and `volatile` accesses of real programs. "Sites" are monitor enters and exits and `volatile` accesses outside the virtual machine's own classes. "Removable" are those whose receivers are all thread-local. AWFY and javac are from 24 and 28 October; the rest is from 29 October.

Program	Sites	Removable
AWFY (allocation sites)	1,061	29, all in the JDK; the phase removes 58 locks and makes 10 accesses ordinary; none of AWFY's own
javac, default cutoff	3,707	0
javac, cutoff 100	3,707	66; 16 locks (168 operations) removed, 23 accesses made ordinary, all in the JDK; no change in run time
PetClinic	17,496	0
11 Barista servers	1,303–10,721 per server	0 in every export

6.4.2 Real Programs

Table 6.8 gives the locks of real programs. The answer is the same in all of them: nothing the programs own is provably thread-local.

AWFY has no lock of its own, so there is nothing to measure. On javac the saturation cutoff, the point at which Native Image gives up the types of a flow and treats it as any type, decides the answer: at the default of 20 no site is provable, and at 100 there are 66, from which the build removes 16 locks. Compiling AWFY's 79 sources then takes 0.240 seconds against 0.239: the locks removed are in the JDK, javac is mostly one thread, and nothing is gained. On PetClinic every one of the 9,729 monitor sites and 7,767 `volatile` sites has a shared object among its receivers, receivers the analysis cannot name, or none at all. The request threads share the database, the connection pool, and the caches, so the locks *are* on shared state, and the analysis says so correctly. Throughput does not tell the builds apart either: the same binary repeats within 0.2–0.9 %, but four builds of the same configuration span 6 % for `graal` and 38 % for `kills`, with identical kill sets and code size, so no effect of the facts can be read from PetClinic. Of Barista's 13 applications, the two Quarkus ones failed to build with and without the feature, which we read, but cannot establish, as a mismatch between the bundles and our builder.

6.4.3 Why No Lock Is Provable

We took javac's export apart with controls that leave one kind of publishing site out of thread escape; these are unsound and used only for diagnosis. Each of the three ways a reference becomes shared (a store into a static field, a field, or an array) rebuilds nearly the whole shared set on its own (about 220,000 names), and only leaving out all of them collapses it (to 5,773 names, with 1,230 provable sites). What seeds the closure is saturation: a flow that exceeds the cutoff is rebuilt

as "any `Object`", and an `Object` that is shared is everything. A larger cutoff halves the shared set and makes the 66 sites provable; most of what remains are stores `this.f = v` into a field of a class, which cannot say into which instance they store. On the Vert.x server the connection looks like a hand-off from the acceptor to an event-loop thread, but the trace refutes this: it is shared by array aliasing, one elements flow for all arrays, and 57 % of the 277,782 shared objects root at one static array in `BigDecimal`. Leaving out the stores into arrays alone changes nothing; leaving out every kind of store would free 2,773 of 5,917 sites. We conclude that the limit is the precision of Native Image's own points-to analysis, not the thread model.

6.5 Soundness in Practice (RQ4)

Soundness is not proved here for the whole pipeline; Chapter 4 proves it for the model, and the proofs are sketches in three places. The evidence of this chapter is of another kind, and it has three parts.

Every image prints what the JVM prints. This holds for 28 round-trip images, 159 images of the example suite, the concurrency round trips, and AWFY with lock elision.

Controls that must fail do. A claim that is false must produce a wrong result, or the test cannot see it. A method claimed to write nothing gives 171 instead of 177 in `Examples` at `-O3`. A shared lock claimed local gives 317,288 instead of 800,000, or a hang; a flag read claimed ordinary hangs the image. One control was weak: the first claim that references which may meet are apart still printed the right value, because a wrong ordering shows only when Graal forwards a wrong value. How to catch a *missing* ordering remains open.

Native Image's own tests. With the feature and `-J-ea -J-esa`, the 23 images of its JUnit suite build and all 289 tests pass. The suite exercises lock elision not at all (there is nothing to remove in any image) and dead stores barely, so it shows that kill sets and partitions do not break Native Image, and that the phases build under assertions. The assertions also found three defects that the examples did not: disabled flows asked for their objects, calls killed the initialization state, and a JSON escape failed on half a surrogate pair. With allocation sites alone and no feature at all, the JMX tests fail, so the allocation-site analysis cannot be the base of this check.

6.6 What Blocks the Analysis

The gain on AWFY is small because most of its methods remain opaque. Of the 459 methods of the program, 199 still kill everything. Table 6.9 shows what freeing each reason alone would do.

Table 6.9: Methods of AWFY (of 459) that still kill everything when the kill of the named effect is left out of the kill sets (22 October export). Left out together, the effects free all 199.

Left out	Methods freed
identity hash codes, synchronization, unresolved calls, or other effects, each alone	0
class initialization alone	12
identity hash, class initialization, and synchronization together	12
… and unresolved calls	12
… and every other effect	199

No single reason is enough. Each of the 187 methods that remain after the first three reaches all of an unresolved call, a transition to native code, a raw write, and a snippet, and the reasons form a chain: a method that may build an exception, or call `equals` on an object of unknown type, reaches a lock, then class initialization, then a native call. Modelling concurrency removes the first layer only. This is the reason for the ceiling result of Section 6.2: the layers have to go together, and even then AWFY does not run faster.

6.7 Discussion

RQ1. Graal optimizes a little more with the facts, and in a specific way. Where it must assume that a call writes anything and the call sits in a loop, the kill set lets loads leave the loop; Queens is that case, with −26 % to −28 % in every build. Elsewhere the gain is a handful of loads (2–4 % of AWFY's) that does not show in the total beyond 0 to 3 %. Two results say that the small size is not a weakness of the implementation: the ceiling removes three times the loads and is no faster, and the opacity of AWFY has several layers, each of which would have to go. Partitions by allocation site and dead stores add nothing on AWFY and little on the designed programs. With the same facts, Graal's own phases removed at least as many loads as the rules on the token form, so the token form serves as the model that shows the facts to be correct (Chapter 4), not as a second optimizer.

RQ2. The facts are cheap where they are small: the export takes 2 seconds of an AWFY build and the tool 0.3, and kill sets cost 7–9 % of the build on AWFY and 13–15 % on javac. The cost that matters is the analysis that partitions and decides locks need: allocation sites multiply the build of javac by 4.3 and that of PetClinic by 15, while our own steps stay at 5–6 %. We believe that the idea of explicit effects is therefore affordable as kill sets and expensive as partitions and lock elision, and that the expense belongs to the finer analysis.

RQ3. The mechanism works and is sound: thread escape proves a reference local where it should and not where it should not, and where it does, the effect is large (16 to 19 times for a lock, 8 for

a `volatile` write). On real programs it proves nothing about the locks they own, and the reasons are in the points-to analysis: array aliasing, saturated flows that become "any `Object`", and field stores that cannot name their instance. Raising the cutoff moved javac, and moved nothing in its speed. The honest statement is that no real program in the set benefits today. We do not claim that none would: a program whose locks guard thread-local objects that the analysis can name was not among the eleven servers, PetClinic, javac, and AWFY.

RQ4. Every image printed what the JVM printed, every false claim produced a wrong or hanging image, and Native Image's own tests pass with assertions on. This is evidence by controls and examples, not a proof; the assumptions of thread escape (`vm-code` and `entry-points`) are not proved, and one of them once hid a real publication, which the list of removable sites of AWFY revealed.

Which baseline. The comparison of `graal` with `kills` on the default analysis is the main one: it is what a user gets today, at ordinary build cost, with nothing else changed. The same comparison on allocation sites shows that the result does not depend on the analysis (the same 40 loads). The comparisons of `kills` with `objects` and of the flags baseline with `locks` are the effect of the partitions and of lock elision, each on allocation sites. For the partitions this is forced. For lock elision it is a choice: we timed it only on allocation sites, and on the default analysis we built it only for the Barista servers, where it removed nothing. The matrix gives the same verdicts under both analyses for the four plain cases, so we expect the shapes to behave alike, but we have not shown it. A comparison across the two analyses measures the analysis, not our facts, and we use it only once, to show why it is not used.

6.8 Threats to Validity

One machine and few programs. Every timing is from one server with an AMD Zen 1 processor, without hardware counters, and the placement analysis was done earlier on another machine. The programs are AWFY, javac, designed shapes, PetClinic, and the servers of one suite; the gains are on designed shapes, and no real program shows a lock removed that matters or a run-time gain beyond Queens.

Code placement. Placement moves run time by 4–13 % with identical instructions, which is above the effect of the facts for all benchmarks but Queens. We address it by timing twelve builds, and we do not cite the benchmarks that vary between builds. Framework images vary more than binaries: four PetClinic builds of one configuration differ in speed by up to 38 % with identical kill sets.

Static counts. A load left is not a load executed, and loop unrolling multiplies counts. Native Image also specializes on its closed world, so very small programs behave unlike real ones: a literal trip count unrolled a loop completely, and a null argument let it prove that a call always throws.

Lock elision was timed on one analysis. The run times of Table 6.7 are from allocation sites. The default analysis finds fewer sites (34 against 43 thread-local in AWFY) and we did not time it, so we do not know how much of the 16 to 19 times would remain there.

The baseline moves. The build time of the allocation-site analysis varies by 30 seconds between builds of javac, so we quote only our own steps for it. The measurements of 22–28 October were made from stamped working copies, not from commits; the clean run of 29 October confirmed their loads to the unit, their shapes to the second decimal, and their costs within the spread. The clean run of 31 October confirmed the first in turn: the loads to the unit, the shapes to 0.1 millisecond, and the costs within the spread, with two exceptions. `graal`'s image of the field lock was 17 % slower, and the cost of the partitions was 11 % instead of 4 %; we report both as ranges. The total run-time gain of kill sets was not the same in the three sets of builds (means −2.3 %, −0.4 %, and −1.9 %), which is why we report the range of twelve builds.

Assumptions and proofs. The assumptions of thread escape are not proved, and the soundness proofs are sketches in three places. The weak control for a missing ordering, mentioned above, is a gap in what the tests can see. The experiment with Native Image's layered images ran the feature in a base layer and crashed in the application layer, so we make no claim about analyzing a shared library once.

7 Related Work

7.1 Side-Effect and Mod/Ref Analyses

7.2 Memory in Compiler Representations

7.3 Escape Analysis and Synchronization Removal

7.4 Analyses for Ahead-of-Time Compilation

8 Conclusion

8.1 Results

8.2 Contribution

8.3 Future Work

Bibliography

[1] *Barista Benchmark Suite*. https://github.com/barista-benchmarks/barista. Version 0.6.6.

[2] Stefan Marr, Benoit Daloze, and Hanspeter Mössenböck. "Cross-Language Compiler Benchmarking: Are We Fast Yet?" In: *Proceedings of the 12th Symposium on Dynamic Languages (DLS)*. 2016.

A Electronic Attachments

B Proofs

C The Dialect of Graal IR in Cthulhu

D Reproducing the Results

This appendix lists what is needed to regenerate the tables of Chapter 6. All commands are in the directories `scripts` and `graal-native` of the tool's repository; the builder is the Graal fork, which the repository contains as a submodule.

D.1 The Headline Tables from One Commit

The script `reproduce.sh` produces the tables of Sections 6.2 and 6.3 and of the lock shapes in Section 6.4 in one run. It refuses to start unless the tool and the fork are exactly a commit (no changed tracked files, no untracked files) and the builder is not older than the fork's sources. With `ALLOW_DIRTY=1` it runs for testing and marks the results `NOT-REPRODUCIBLE`. The procedure of the clean runs of 29 and 31 October, on a machine where nothing else runs, was:

```
# on the development machine, from clean, committed checkouts of the tool and of the fork:
scripts/server/sync-pontos.sh      # copies both trees to the server and stamps them with their commits

# on the server:
export JAVA_HOME=$HOME/.mx/jdks/labsjdk-ce-latest-jvmci-25.4-b23_amd64
export PATH=$JAVA_HOME/bin:$HOME/mx:$HOME/.local/bin:$PATH
cd ~/graal/substratevm && mx build # the builder must be newer than the sources
~/explicit-effects/scripts/evaluation/reproduce.sh
```

In `graal-native`, the results are written to `out/reproduction/<UTC time>-<commit>/`: `provenance.txt` (which code), one file per stage, and `SUMMARY.txt`. Table D.1 lists the stages. The default stages took 74 minutes on 29 October and 75 on 31 October.

D.2 Other Measurements

The results that `reproduce.sh` does not regenerate have their own scripts, listed in Table D.2. The default stages of the two clean runs reproduced the loads of Table 6.3 to the unit, and the lock shapes of Table 6.7 to 0.1 millisecond except one baseline, which moved by 17 %; the matrix, javac, PetClinic, and the Barista applications were not part of them.

Table D.1: The stages of `reproduce.sh`. The first five run by default; the last three are long and run with `reproduce.sh matrix javac frameworks` or `reproduce.sh all`.

Stage	Section	What it runs
`loads`	6.2	the three configurations on the four small programs and on AWFY at `-O2`, on both analyses, and the counts
`time`	6.2	`graal` and `kills` on AWFY at `-O3`, four builds each, timed together with the A/A control
`ceiling`	6.2	`graal`, `kills`, and `ceiling` on AWFY at `-O3`: loads left and run times
`cost`	6.3	every configuration of AWFY, five rounds, with the stage times and the peak memory
`shapes`	6.4	the lock and `volatile` shapes, `graal` against elision
`matrix`	6.4	thread escape on the concurrency examples, every analysis and every set of assumptions
`javac`	6.3, 6.4	javac with the cutoff and lock removal, three rounds, and the time it takes to compile
`frameworks`	6.3, 6.4	PetClinic and three Barista servers: builds, removable sites, PetClinic under load

Table D.2: Scripts for the results outside the default stages.

Result	Script
Barista, the other ten applications	`barista-gate.sh`; the bundles are made with Barista's `./build <application>`
PetClinic builds and load	`petclinic.sh`, `petclinic_load.py`; the 2023 bundle of Graal's benchmark suite with four reflection registrations
Why no lock is removable	`why_no_locks.py <export.json>`, which uses the decision code of the tool; on javac's export it finds the 66 sites the tool lists
Native Image's JUnit tests	`mx native-unittest com.oracle.svm.test` with the feature and `-J-ea`
Placement of code	`programs/awfy/layouts.py` and `time.py`
Controls	`LIE=<method> ./run.sh Examples`, `LIE_LOCKS=<method> ONLY=locks ./run.sh LocksShared`, `SPLIT=<method>`; each must go wrong